Сетевая архитектура

Сетевая архитектура

Проблема

Допустим, вы сделали игру. Один игрок, враги, коллизии, анимации - всё работает. А теперь друг хочет играть вместе с вами. Два компьютера, один мир. Как?

Банальная мысль: каждый кадр сервер отправляет каждому клиенту **полное состояние мира** - позиции всех объектов, здоровье, состояния автоматов, таймеры анимаций. Считаем: 100 объектов × 50 байт = 5 КБ на кадр. При 60 FPS это 300 КБ/с на одного клиента. "Пф, да современные компы круты!".
А если объектов 1000 по 200 байт? 200 КБ на кадр × 60 FPS = 12 МБ/с на одного клиента. При 10 игроках сервер рассылает это каждому: 120 МБ/с исходящего трафика. Это уже серьёзно.

И это только половина проблемы. Вторая: **задержка**. Пакет летит от Москвы до Новосибирска ~50 мс. До Нью-Йорка - 100+ мс. Если мы рисуем мир только когда получили новые данные, игра будет реагировать на ввод с задержкой в 100 мс. Для шахмат это нормально. Для шутера - невыносимо.

Т.е. нам нужно решить две задачи:

1. **Что отправлять** - как компактно упаковать данные (сериализация).
2. **Как скрыть задержку** - как сделать игру отзывчивой несмотря на пинг (предсказание).

Но сначала - как вообще два компьютера обмениваются данными.

Сокеты: основы

Сокет - это точка соединения. Грубо говоря, труба между двумя программами, через которую летят байты. Один компьютер создаёт сокет, привязывает его к адресу, другой подключается.

Что происходит на низком уровне

Ну, когда вы пишете `sock.sendto(b"hello", addr)`, происходит примерно следующее:

1. **Байты.** Строка `"hello"` - это 5 байт: `68 65 6C 6C 6F` или 40 бит: 01101000 01100101 01101100 01101100 01101111 (ASCII-коды). Всё, что летит по сети - это последовательности нулей и единиц, больше ничего.
2. **Упаковка.** ОС оборачивает ваши байты в **пакет**: добавляет заголовки - IP-адрес отправителя, IP-адрес получателя, порт, длину данных, контрольную сумму. Получается что-то вроде конверта с адресом и письмом внутри.

3. **Передача.** Сетевая карта превращает пакет в электрические сигналы (Ethernet), радиоволны (Wi-Fi) или световые импульсы (оптоволокно). Каждый бит - высокое или низкое напряжение, наличие или отсутствие сигнала.
4. **Маршрутизация.** Пакет проходит через роутеры - каждый читает IP-адрес назначения и решает, куда пакет отправить дальше. Как почтовые отделения пересылают письмо от города к городу.
5. **Приём.** Сетевая карта получателя собирает сигналы обратно в биты, ОС проверяет контрольную сумму (не повредился ли пакет), находит по номеру порта нужную программу и кладёт данные в **буфер** сокета. Когда ваш код вызывает `recv()` - он просто читает из этого буфера.

Т.е. `sendto` и `recv` - это не прямая связь между программами. Это работа с буферами ОС, а всю доставку берёт на себя сетевой стек: ОС, драйвер сетевой карты, роутеры.

Контрольная сумма - это число, вычисленное из содержимого пакета. Получатель пересчитывает её и сравнивает. Если не совпало - пакет повреждён. TCP перезапрашивает его, UDP просто выбрасывает.

IP и порт

Адрес сокета - это **IP + порт**. IP - адрес компьютера в сети (`192.168.1.5` , `127.0.0.1` - это localhost, т.е. ваш же компьютер). Порт - число от 0 до 65535, определяющее, какой именно программе на этом компьютере предназначены данные. Т.е. IP - это адрес дома, порт - номер квартиры.

`0.0.0.0` в коде сервера означает "слушать на всех сетевых интерфейсах" - и по localhost, и по Wi-Fi, и по Ethernet.

TCP и UDP

Два протокола:

- **TCP** - устанавливает соединение (connect → accept), гарантирует доставку и порядок. Если пакет потерялся - перешлёт. Удобно, но добавляет задержку.
- **UDP** - без соединения. Отправляет пакет (datagram) и забывает. Быстрее, но пакет может потеряться или прийти не в том порядке.

Для реалтайм-игр обычно используют UDP (скорость важнее гарантии). Для пошаговых - TCP (надёжность важнее скорости).

TCP-пример

Начнём с TCP - он проще для понимания и подходит для первого прототипа. Сервер слушает, клиент подключается, дальше оба могут слать и получать данные.

Python C#

```python
import socket

# --- Server ---

```

```python
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP
server.bind(("0.0.0.0", 5555))
server.listen(1)  # queue of 1 pending connection

conn, addr = server.accept()  # blocks until client connects
print(f"Client connected from {addr}")

data = conn.recv(1024)
print(f"Got: {data.decode()}")
conn.sendall(b"hello back")

conn.close()
server.close()


# Client
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5555))

client.sendall(b"hello server")
response = client.recv(1024)
print(f"Server says: {response.decode()}")

client.close()
```

Ключевое отличие от UDP: сначала `connect` / `accept` , потом обмен данными. TCP гарантирует, что `sendall` дойдёт целиком и в правильном порядке. Для пошаговой игры, где ходы летают раз в несколько секунд, этого более чем достаточно.

UDP-пример

Для реалтайма нужен UDP - без установки соединения, без ожидания подтверждения:

Python C#

```python
import socket

# Server
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # UDP
server.bind(("0.0.0.0", 5555))

data, addr = server.recvfrom(1024)
print(f"Got {data} from {addr}")
server.sendto(b"hello back", addr)


# Client
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto(b"hello server", ("127.0.0.1", 5555))

response, _ = client.recvfrom(1024)
print(f"Server says: {response}")
```

Обратите внимание: нет `connect`, нет `accept`. Сервер просто слушает порт, клиент шлёт пакет на адрес. Каждый пакет независим. Потерялся - ну и ладно, через 16 мс придёт следующий с актуальными данными.

Неблокирующие сокеты в игровом цикле

Оба примера выше **блокирующие**: `recv` / `recvfrom` останавливают программу, пока не придут данные. В игровом цикле это недопустимо - цикл должен работать каждый кадр.

Решение - **неблокирующий режим**: сокет сразу возвращает управление, даже если данных нет.

Python | C#

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", 5555))
sock.setblocking(False)  # non-blocking mode

# inside game loop:
def network_update():
    while True:
        try:
            data, addr = sock.recvfrom(1024)
            handle_packet(data, addr)
        except BlockingIOError:
            break  # no more packets this frame
```

`while True` + `except BlockingIOError` - классический подход: читаем все пакеты, накопившиеся с прошлого кадра, и выходим, когда больше нечего читать. Никаких зависаний, цикл продолжает работать.

> Альтернатива неблокирующим сокетам - **отдельный поток** (`threading.Thread` / `Task.Run`). Поток читает сокет в цикле и складывает полученные пакеты в потокобезопасную очередь, а игровой цикл каждый кадр забирает из неё.

Сериализация

Сокеты передают байты. Нам нужно превратить игровые данные (позицию, здоровье, состояние) в нолики и единички, отправить, а на другом конце - превратить обратно.

Это и есть **сериализация** (данные -> байты) и **десериализация** (байты -> данные).

Наивный подход: JSON

Первая мысль - `json.dumps`. Это работает, но неэффективно. JSON - формат текстовый: число `1920` занимает 4 байта как текст, но всего 2 байта как значение типа `short`. При 60 пакетах в секунду каждый лишний байт идет лишним грузом.

Бинарная упаковка: struct

`struct` (Python) и `BitConverter` / `BinaryWriter` (C#) упаковывают данные в минимальное число байт.

Python | C#

```python
import struct

def serialize_player(player):
    # pack: float x, float y, int hp, byte state
    return struct.pack("!ffIB",
                       player.x, player.y, player.hp, player.state)

def deserialize_player(data):
    x, y, hp, state = struct.unpack("!ffIB", data)
    return {"x": x, "y": y, "hp": hp, "state": state}


# test
raw = serialize_player(player)
print(f"Size: {len(raw)} bytes")  # 13 bytes (4+4+4+1)

restored = deserialize_player(raw)
```

13 байт вместо ~50 в JSON. При 100 объектах - 1.3 КБ вместо 5 КБ. При 60 FPS - 78 КБ/с вместо 300 КБ/с.

Формат `"!ffIB"` в Python struct: `!` = network byte order (big-endian), `f` = float (4 bytes), `I` = unsigned int (4 bytes), `B` = unsigned byte (1 byte). В C# `BinaryWriter` пишет в **little-endian**. Т.е. Python и C# из этих примеров напрямую несовместимы - для кросс-языкового взаимодействия нужно согласовать порядок байт (например, использовать `IPAddress.HostToNetworkOrder` в C# или `"<"` в Python struct).

Дельта-компрессия

Ещё эффективнее - отправлять не полное состояние, а только **что изменилось** с прошлого кадра. Если из 100 объектов за кадр изменились 5 - отправляем данные только 5-ти.

Python C#

```python
def make_delta(prev_state, curr_state):
    """Return only changed fields."""
    delta = {}
    for entity_id, entity in curr_state.items():
        prev = prev_state.get(entity_id)
        if prev is None:
            delta[entity_id] = entity  # new entity
        elif entity != prev:
            delta[entity_id] = entity  # changed
    return delta


def apply_delta(state, delta):
    """Apply delta to state."""
    for entity_id, entity in delta.items():
        state[entity_id] = entity
    return state
```

Дельта-компрессия - стандартный подход в сетевых играх. Quake 3 отправляла ~10-20 КБ/с на клиента вместо сотен КБ/с именно благодаря дельтам.

Клиент-сервер

Ок. Мы умеем упаковывать данные и отправлять по сети. Но **кто главный**? Если два игрока одновременно двигаются - кто решает, где каждый стоит?

Наивный подход: доверяй клиенту

Каждый клиент считает свою физику локально и говорит серверу: "Я стою в точке (300, 200)". Сервер пересылает эту информацию другим клиентам.

Проблема? **Читы**. Клиент может сказать: "Я стою на вражеской базе" или "Моё здоровье - 999999". Если сервер доверяет клиенту - читер неуязвим.

Авторитарный сервер

В правильной архитектуре **сервер - авторитет**. Клиент не говорит "я в точке X". Клиент говорит "я нажал W" (отправляет ввод). Сервер сам считает физику, коллизии, урон - и отправляет клиентам **результат**.

Python C#

```python
# --- Client sends only input ---
def client_tick(sock, server_addr):
    keys = get_pressed_keys()
    input_data = struct.pack("!BBB",
                             keys["up"], keys["down"], keys["shoot"])
    sock.sendto(input_data, server_addr)


# --- Server processes input, sends state ---
def server_tick(sock, clients, world):
    # read all pending packets (socket is non-blocking)
    while True:
        try:
            data, addr = sock.recvfrom(64)
            if addr not in clients:
                continue
            up, down, shoot = struct.unpack("!BBB", data)
            player = world.get_player(addr)
            if up:
                player.y -= SPEED * DT
            if down:
                player.y += SPEED * DT
            if shoot:
                world.spawn_bullet(player)
        except BlockingIOError:
            break  # no more packets this tick

    # physics, collisions - all on server
    world.update(DT)

    # send world state to all clients
    state = serialize_world(world)
    for addr in clients:
        sock.sendto(state, addr)
```

Клиент отправляет 3 байта (нажатия клавиш). Сервер считает всё сам и рассылает результат. Читер может врать про ввод, но не может телепортироваться - сервер проверяет физику. К тому же, ввод мы тоже можем проверять. Если человек провел мышкой со скоростью 100м/с - наверное, что-то не так.

Авторитарный сервер - стандарт для шутеров (CS:GO, Valorant, Overwatch). Для кооперативных игр без PvP можно ослабить контроль - доверять клиентам частично.

Предсказание и коррекция

А вот тут начинается самое интересное.

Проблема задержки

Клиент нажимает W. Отправляет ввод на сервер. Сервер обрабатывает, отправляет обратно новое состояние. **Round-trip time (RTT)** - 80 мс. Игрок нажимает W и видит, что персонаж начинает двигаться через 80 мс. Ощущение: "игра тормозит, как через воду".

Решение: предсказание на клиенте

А зачем клиенту **ждать** ответ сервера. Он может сам симулировать результат нажатия - сразу двигать персонажа локально. А когда приходит ответ от сервера, проверяет: совпало или нет. Читер, конечно, может попытаться себя обмануть. Но ничего, что повлияет на других, сделать он не сможет. Да и увидеть что-то, чего он не получил от сервера, тоже.

Идея по шагам:

1. Игрок нажимает W -> клиент **сразу** двигает персонажа (предсказание).
2. Клиент сохраняет этот ввод в `pending_inputs` с порядковым номером.
3. Клиент отправляет ввод на сервер.
4. Сервер получает ввод, применяет, отправляет обратно: "Ты в точке (x, y), я обработал ввод #42".
5. Клиент получает ответ. Выбрасывает все inputs до #42 (они уже подтверждены). Ставит позицию с сервера. **Переприменяет** оставшиеся неподтверждённые inputs (#43, #44...).

В результате: игрок видит мгновенную реакцию (локальное предсказание), но сервер остаётся авторитетом. Если сервер не согласен (например, на пути была стена, про которую клиент не знал), клиент "откатывается" к серверной позиции.

По сути, **client-side prediction with server reconciliation**.

А что видят другие игроки?

Вы предсказываете свою позицию. Но позиции других игроков приходят с сервера с задержкой. Если рисовать их ровно там, где сказал сервер, они будут дёргаться - пакеты приходят неравномерно.

Решение - **интерполяция**: рисовать чужих игроков **на 100 мс в прошлом**, плавно перемещая между двумя последними известными позициями. Игрок этого не замечает - зато движение плавное, без рывков.

Интерполяция означает, что чужие игроки **всегда немного в прошлом**. Когда вы стреляете в другого игрока, ваш прицел наведён на позицию, где он был 100 мс назад. Для компенсации в шутерах используют **lag compensation** - сервер "отматывает время" и проверяет попадание по исторической позиции. Это отдельная тема, но принцип понятен: та же идея кольцевого буфера, что и в FPS-счётчике.

А если игра пошаговая?

В пошаговых играх проблема проще. Нет реалтайма - нет проблемы с задержкой. Игрок думает 10 секунд, нажимает "ход", команда летит на сервер, сервер выполняет, рассылает результат. 80 мс задержки на фоне 10 секунд - незаметно.

Для пошаговых игр идеально подходит **lockstep**: все клиенты получают одни и те же команды и выполняют их синхронно. Не нужно пересылать состояние мира - только команды. Файл реплея весит килобайты.

	Realtime	Turn-based
Протокол	UDP (скорость)	TCP (надёжность)
Синхронизация	Авторитарный сервер + предсказание	Lockstep (обмен командами)
Задержка критична?	Да (нужно предсказание)	Нет (игрок думает секунды)
Читы	Сервер проверяет всё	Детерминизм + хеш-верификация
Пример	CS:GO, Overwatch	Age of Empires, Civilization

Итого

Сетевая архитектура - это три слоя (хотя в целом, и поболее можно придумать):

1. **Сериализация** - как упаковать данные в байты. `struct.pack` / `BinaryWriter`, дельта-компрессия.
2. **Авторитарный сервер** - клиент отправляет ввод, сервер решает. Защита от читов.
3. **Предсказание + коррекция** - клиент предсказывает локально, сервер корректирует, клиент переприменяет неподтверждённые inputs.

Для проекта: если делаете кооп - начните с TCP-сервера, который пересылает состояние. Когда это заработает, попробуйте дельта-компрессию. Предсказание - продвинутый уровень, но если дойдёте до него, это серьёзное усиление проекта.

Что	Где подробнее
Паттерн Command для lockstep	Пошаговые игры
Конечные автоматы (состояния игрока на сервере)	Конечные автоматы
Кольцевой буфер (FPS-счётчик, история позиций)	Отладка
deltaTime и игровой цикл на сервере	Основной цикл
Пространственные структуры (серверная физика)	Пространственные структуры

Powered by Wiki.js